What is version control?

5 minutes

A version control system (VCS) is a program or set of programs that tracks changes to a collection of files. One goal of a VCS is to easily recall earlier versions of individual files or of the entire project. Another goal is to allow several team members to work on a project, even on the same files, at the same time without affecting each other's work.

Another name for a VCS is a software configuration management (SCM) system. The two terms often are used interchangeably—in fact, Git's official documentation is located at git-scm.com . Technically, version control is just one of the practices involved in SCM. A VCS can be used for projects other than software, including books and online tutorials.

With a VCS, you can:

- See all the changes made to your project, when the changes were made, and who made them.
- Include a message with each change to explain the reasoning behind it.
- Retrieve past versions of the entire project or of individual files.
- Create *branches*, where changes can be made experimentally. This feature allows several different sets of changes (for example, features or bug fixes) to be worked on at the same time, possibly by different people, without affecting the main branch. Later, you can merge the changes you want to keep back into the main branch.
- Attach a tag to a version—for example, to mark a new release.

Git is a fast, versatile, highly scalable, free, open-source VCS. Its primary author is Linus Torvalds, the creator of Linux.

Distributed version control

Earlier instances of VCSes, including CVS, Subversion (SVN), and Perforce, used a centralized server to store a project's history. This centralization meant that the one server was also potentially a single point of failure.

Git is *distributed*, which means that a project's complete history is stored both on the client *and* on the server. You can edit files without a network connection, check them in locally, and sync

with the server when a connection becomes available. If a server goes down, you still have a local copy of the project. Technically, you don't even have to have a server. Changes could be passed around in e-mail or shared by using removable media, but no one uses Git this way in practice.

Git terminology

To understand Git, you have to understand the terminology. Here's a short list of terms that Git users frequently use. Don't be concerned about the details for now; all these terms will become familiar as you work your way through the exercises in this module.

- **Working tree**: The set of nested directories and files that contain the project that's being worked on.
- **Repository (repo)**: The directory, located at the top level of a working tree, where Git keeps all the history and metadata for a project. Repositories are almost always referred to as *repos*. A *bare repository* is one that isn't part of a working tree; it's used for sharing or backup. A bare repo is usually a directory with a name that ends in *.git*—for example, *project.git*.
- **Hash**: A number produced by a hash function that represents the contents of a file or another object as a fixed number of digits. Git uses hashes that are 160 bits long. One advantage to using hashes is that Git can tell whether a file has changed by hashing its contents and comparing the result to the previous hash. If the file time-and-date stamp is changed, but the file hash isn’t changed, Git knows the file contents aren’t changed.
- **Object**: A Git repo contains four types of *objects,* each uniquely identified by an SHA-1 hash. A *blob* object contains an ordinary file. A *tree* object represents a directory; it contains names, hashes, and permissions. A *commit* object represents a specific version of the working tree. A *tag* is a name attached to a commit.
- **Commit**: When used as a verb, *commit* means to make a commit object. This action takes its name from commits to a database. It means you are committing the changes you have made so that others can eventually see them, too.
- **Branch**: A branch is a named series of linked commits. The most recent commit on a branch is called the *head*. The default branch, which is created when you initialize a repository, is called `main`, often named `master` in Git. The head of the current branch is named `HEAD`. Branches are an incredibly useful feature of Git because they allow developers to work independently (or together) in branches and later merge their changes into the default branch.

- **Remote**: A remote is a named reference to another Git repository. When you create a repo, Git creates a remote named `origin` that is the default remote for push and pull operations.

- **Commands**, **subcommands**, and **options**: Git operations are performed by using commands like `git push` and `git pull`. `git` is the command, and `push` or `pull` is the subcommand. The subcommand specifies the operation you want Git to perform. Commands frequently are accompanied by options, which use hyphens (-) or double hyphens (--). For example, `git reset --hard`.

These terms and others, like `push` and `pull`, will make more sense shortly. But you have to start somewhere, and you might find it helpful to come back and review this glossary of terms after you finish the module.

The Git command line

Several different GUIs are available for Git, including GitHub Desktop. Many programming editors, like Microsoft Visual Studio Code, also have an interface to Git. They all work differently and they have different limitations. None of them implement *all* of Git's functionality.

The exercises in this module use the Git command line—specifically, Git commands executed in Azure Cloud Shell. However, Git's command-line interface works the same, no matter what operating system you're using. Plus, the command line lets you tap into *all* of Git's functionality. Developers who see Git only through a GUI sometimes find themselves confronted with error messages they can't resolve, and they have to resort to the command line to get going again.

Git and GitHub

As you work with Git, you might wonder about differences between the features it offers and the features offered on GitHub.

As mentioned earlier, Git is a distributed version control system (DVCS) that multiple developers and other contributors can use to work on a project. It provides a way to work with one or more local branches and then push them to a remote repository.

GitHub is a cloud platform that uses Git as its core technology. GitHub simplifies the process of collaborating on projects and provides a website, more command-line tools, and overall flow that developers and users can use to work together. GitHub acts as the remote repository mentioned earlier.

Key features provided by GitHub include:

- Issues
- Discussions
- Pull requests
- Notifications
- Labels
- Actions
- Forks
- Projects

To learn more about GitHub, see the Introduction to GitHub Microsoft Learn module or the Getting started with GitHub help documentation.

The next step is to try out Git for yourself!

Next unit: Exercise - Try out Git

Previous Next